

Testes e Exceções em Python

Aula 14 e 15

Ana Júlia Lima



Desenvolvido para
Young 1 - sexta 8001

Ctrl Play
Brasil

Sumário

1	Testes em Python (unittest)	2
1.1	Por que testar?	2
1.2	O que são testes?	2
1.3	Como pensar em testes (AAA)	2
1.4	Testes de unidade com <code>unittest</code>	2
1.5	Como vemos a função passando no teste?	2
1.6	Testando funções	2
1.7	Como testar uma classe? <code>setUp()</code>	3
1.8	Boas práticas rápidas	4
2	Tratamento de Exceções em Python	4
2.1	O que são exceções?	4
2.2	Por que tratar exceções?	4
2.3	Erros comuns em Python	5
2.4	<code>try</code> , <code>except</code> e <code>else</code>	5
2.5	<code>finally</code>	5
2.6	Resumo rápido	5
3	Atividades — Testes e Exceções	6
4	Desafio Final — Sistema Seguro	7
5	Gabarito — Testes e Exceções	8
6	Gabarito — Desafio Final	12
7	Mini Projeto Final — Sistema de Cadastro Seguro (Nível Prova)	13
7.1	Objetivo do sistema	13
7.2	Arquivo principal (<code>sistema.py</code>)	13
7.3	Interface com tratamento de erros (<code>main.py</code>)	14
7.4	Testes automatizados (<code>test_sistema.py</code>)	15
7.5	Exemplo de uso	15
7.6	O que este projeto trabalha	16
7.7	Desafios extras (nível hard)	16

1 Testes em Python (unittest)

1.1 Por que testar?

Testes são pequenos programas que exercitam o seu código com entradas controladas e verificam se a saída está correta. Eles evitam regressões, documentam o comportamento esperado e dão confiança para refatorar.

1.2 O que são testes?

Em Python, a biblioteca padrão `unittest` fornece:

- **Casos de teste** (`TestCase`): agrupam métodos de teste;
- **Asserts**: verificam resultados;
- **Runner**: executa e relata o status dos testes.

1.3 Como pensar em testes (AAA)

Siga o padrão **Arrange–Act–Assert**:

1. **Arrange**: preparo dos dados/objetos;
2. **Act**: chamada da função/método sob teste;
3. **Assert**: verificação com asserts.

Cubra casos felizes, bordas (vazio, `None`, extremos), erros esperados e tipos inválidos (quando fizer sentido).

1.4 Testes de unidade com unittest

Crie classes que herdam de `unittest.TestCase`. Cada método que começa com `test_` é um teste. Principais asserts:

- `self.assertEqual(a, b)`, `self.assertNotEqual(a, b)`
- `self.assertTrue(x)`, `self.assertFalse(x)`
- `self.assertIn(item, colecao)`, `self.assertNotIn(item, colecao)`

Dica (notebooks) Use `unittest.main(argv=[''], exit=False)` para evitar sair do interpretador ao finalizar os testes.

1.5 Como vemos a função passando no teste?

Na execução, o runner mostra . para *pass*, F para *fail* e E para *error*. Ao final, um sumário indica quantos testes passaram/falharam, com mensagens e *tracebacks* úteis.

1.6 Testando funções

```
# arquivo: math_utils.py
def somar(a, b):
    return a + b

def eh_par(n):
    return n % 2 == 0
```

```

# arquivo: test_math_utils.py
import unittest
from math_utils import somar, eh_par

class TestMathUtils(unittest.TestCase):
    def test_somar_basico(self):
        # Arrange & Act
        resultado = somar(2, 2)
        # Assert
        self.assertEqual(resultado, 4)
        self.assertNotEqual(resultado, 5)

    def test_eh_par_exemplos(self):
        # Várias entradas em um único teste
        for n, esperado in [(0, True), (1, False), (2, True), (-3, False)]:
            with self.subTest(n=n):
                self.assertEqual(eh_par(n), esperado)

    def test_asserts_varios(self):
        numeros = [1, 2, 3]
        self.assertTrue(eh_par(2))
        self.assertFalse(eh_par(3))
        self.assertIn(2, numeros)
        self.assertNotIn(4, numeros)

if __name__ == '__main__':
    # Em ambientes interativos (ex.: Jupyter), use:
    unittest.main(argv=[''], exit=False)
    # Em scripts normais, pode ser apenas: unittest.main()

```

1.7 Como testar uma classe? setUp()

```

# arquivo: conta.py
class ContaBancaria:
    def __init__(self, titular, saldo_inicial=0.0):
        self.titular = titular
        self.saldo = float(saldo_inicial)

    def depositar(self, valor):
        if valor <= 0:
            raise ValueError("Valor do depósito deve ser positivo.")
        self.saldo += valor

    def sacar(self, valor):
        if valor <= 0:
            raise ValueError("Valor do saque deve ser positivo.")
        if valor > self.saldo:
            raise ValueError("Saldo insuficiente.")
        self.saldo -= valor

# arquivo: test_conta.py
import unittest
from conta import ContaBancaria

class TestContaBancaria(unittest.TestCase):

```

```

def setUp(self):
    # Cria uma nova conta antes de CADA teste (estado limpo)
    self.conta = ContaBancaria("Ana", saldo_inicial=100.0)

def test_depositar_incrementa_saldo(self):
    self.conta.depositar(50.0)      # Act
    self.assertEqual(self.conta.saldo, 150.0)  # Assert

def test_sacar_decrementa_saldo(self):
    self.conta.sacar(40.0)
    self.assertEqual(self.conta.saldo, 60.0)

def test_sacar_maior_que_saldo_lanca_erro(self):
    with self.assertRaises(ValueError):
        self.conta.sacar(999.0)

def test_depositar_valor_invalido_lanca_erro(self):
    for invalido in [0, -10]:
        with self.subTest(valor=invalido):
            with self.assertRaises(ValueError):
                self.conta.depositar(invalido)

if __name__ == '__main__':
    unittest.main(argv=[''], exit=False)

```

1.8 Boas práticas rápidas

- Nomeie testes claramente: `test_acao_condicao_resultado`.
- Um cenário por teste; use `subTest` para variações.
- Evite dependência entre testes; isole com `setUp()`.
- Cubra casos de borda e erros esperados (`assertRaises`).

2 Tratamento de Exceções em Python

2.1 O que são exceções?

Durante a execução de um programa, podem ocorrer situações inesperadas — como tentar dividir por zero, acessar um arquivo inexistente ou converter texto em número. Essas situações geram **exceções**, que são interrupções controladas no fluxo do programa.

2.2 Por que tratar exceções?

- Evita que o programa quebre de forma abrupta;
- Melhora a experiência do usuário;
- Facilita a depuração e manutenção;
- Permite prever comportamentos e garantir estabilidade.

2.3 Erros comuns em Python

- `SyntaxError` — erro de sintaxe (esquecendo `:` ou `)`);
- `ZeroDivisionError` — divisão por zero;
- `ValueError` — tipo de valor incorreto;
- `NameError` — variável inexistente;
- `TypeError` — operação com tipos incompatíveis;
- `IndexError` — índice fora do alcance da lista;
- `FileNotFoundError` — arquivo inexistente.

2.4 `try`, `except` e `else`

O bloco `try` é usado para envolver o código que pode gerar exceções. O bloco `except` captura e trata o erro. O bloco `else` é executado apenas se não ocorrer erro.

```
try:
    n1 = int(input("Digite um número: "))
    n2 = int(input("Digite outro número: "))
    resultado = n1 / n2
except ValueError:
    print("Erro: você precisa digitar apenas números inteiros.")
except ZeroDivisionError:
    print("Erro: não é possível dividir por zero.")
else:
    print(f"Resultado: {resultado}")
```

Se ocorrer erro, o bloco `except` é executado. Se tudo ocorrer bem, o bloco `else` é executado.

2.5 `finally`

O bloco `finally` é executado sempre, independentemente de ter ocorrido erro ou não. É útil para liberar recursos, como fechar arquivos, desconectar banco de dados, etc.

```
try:
    arquivo = open("dados.txt", "r")
    conteudo = arquivo.read()
    print(conteudo)
except FileNotFoundError:
    print("Arquivo não encontrado.")
finally:
    print("Encerrando operação...")
    if 'arquivo' in locals():
        arquivo.close()
```

2.6 Resumo rápido

- `try`: tenta executar um bloco de código;
- `except`: trata o erro, se ocorrer;
- `else`: executa se não houver erro;
- `finally`: executa sempre, com ou sem erro.

3 Atividades — Testes e Exceções

Nível 1 — Conceitos básicos

1. Crie um programa que peça ao usuário dois números inteiros e faça a divisão entre eles.
 - Trate erro de divisão por zero
 - Trate erro caso o usuário digite algo que não seja número
2. Faça um programa que peça um número e mostre se ele é par ou ímpar.
 - Use `try` e `except` para tratar erros de entrada
3. Crie um programa que tente abrir um arquivo chamado `dados.txt`.
 - Se não existir, mostre uma mensagem amigável
 - Use `finally` para mostrar uma mensagem de encerramento

Nível 2 — Trabalhando com exceções

1. Crie um programa que peça a idade do usuário.
 - Se a idade for negativa, lance um erro com `ValueError`
 - Trate esse erro e mostre uma mensagem
2. Faça um programa que percorra uma lista e tente acessar posições inválidas.
 - Use `try/except` para tratar `IndexError`
3. Crie uma função chamada `dividir(a, b)`:
 - Retorne o resultado da divisão
 - Lance erro se `b == 0`
 - Trate o erro fora da função

Nível 3 — Introdução a testes com `unittest`

1. Crie uma função chamada `soma(a, b)`.
 - Escreva um teste com `unittest` verificando:
 - soma de números positivos
 - soma com número negativo
2. Crie uma função chamada `eh_positivo(n)`.
 - Retorne `True` se for positivo
 - Retorne `False` caso contrário
 - Escreva testes usando:
 - `assertTrue`
 - `assertFalse`
3. Crie uma classe `Calculadora` com um método `multiplicar(a, b)`.
 - Escreva um teste usando `unittest`
 - Utilize `setUp()`
4. Crie um teste que use `assertRaises`.
 - Teste uma função que gera erro propositalmente

4 Desafio Final — Sistema Seguro

Crie um programa completo que:

- Peça ao usuário:
 - nome
 - idade
 - número favorito
- Faça:
 - validação de entradas usando `try/except`
 - impedir idade negativa
 - impedir divisão por zero
- Crie uma função para calcular:
 - `100 / número favorito`
- Escreva um teste com `unittest` para essa função

Objetivo: juntar tudo que foi aprendido (entrada, exceções e testes).

5 Gabarito — Testes e Exceções

Nível 1 — Conceitos básicos

1.

```
try:
    n1 = int(input("Digite um número: "))
    n2 = int(input("Digite outro número: "))

    resultado = n1 / n2

except ValueError:
    print("Erro: digite apenas números.")
except ZeroDivisionError:
    print("Erro: não é possível dividir por zero.")
else:
    print("Resultado:", resultado)

# Exemplo de saída:
# Digite um número: 10
# Digite outro número: 2
# Resultado: 5.0
```

2.

```
try:
    numero = int(input("Digite um número: "))

    if numero % 2 == 0:
        print("Par")
    else:
        print("Ímpar")

except ValueError:
    print("Erro: entrada inválida.")

# Saída exemplo:
# Digite um número: 7
# Ímpar
```

3.

```
try:
    arquivo = open("dados.txt", "r")
    print(arquivo.read())

except FileNotFoundError:
    print("Arquivo não encontrado.")

finally:
    print("Encerrando operação...")

# Saída:
# Arquivo não encontrado.
# Encerrando operação...
```

Nível 2 — Trabalhando com exceções

1.

```
try:
    idade = int(input("Digite sua idade: "))

    if idade < 0:
        raise ValueError("Idade não pode ser negativa.")

    print("Idade válida.")

except ValueError as erro:
    print("Erro:", erro)

# Saída:
# Digite sua idade: -5
# Erro: Idade não pode ser negativa.
```

2.

```
lista = [10, 20, 30]

for i in range(5):
    try:
        print(lista[i])
    except IndexError:
        print("Índice inválido!")

# Saída:
# 10
# 20
# 30
# Índice inválido!
# Índice inválido!
```

3.

```
def dividir(a, b):

    if b == 0:
        raise ZeroDivisionError("Divisão por zero não permitida.")

    return a / b

try:
    resultado = dividir(10, 0)
    print(resultado)

except ZeroDivisionError as erro:
    print("Erro:", erro)

# Saída:
# Erro: Divisão por zero não permitida.
```

Nível 3 — Testes com unittest

1.

```

import unittest

def soma(a, b):
    return a + b

class TestSoma(unittest.TestCase):

    def test_soma_positivos(self):
        self.assertEqual(soma(2, 3), 5)

    def test_soma_negativo(self):
        self.assertEqual(soma(-1, 5), 4)

if __name__ == "__main__":
    unittest.main(argv=[''], exit=False)

# Saída:
# ..
# OK

```

2.

```

import unittest

def eh_positivo(n):
    return n > 0

class TestPositivo(unittest.TestCase):

    def test_valores(self):
        self.assertTrue(eh_positivo(10))
        self.assertFalse(eh_positivo(-5))
        self.assertFalse(eh_positivo(0))

if __name__ == "__main__":
    unittest.main(argv=[''], exit=False)

```

3.

```

import unittest

class Calculadora:

    def multiplicar(self, a, b):
        return a * b

class TestCalculadora(unittest.TestCase):

    def setUp(self):
        self.calc = Calculadora()

    def test_multiplicacao(self):
        self.assertEqual(self.calc.multiplicar(2, 3), 6)

if __name__ == "__main__":
    unittest.main(argv=[''], exit=False)

```

4.

```
import unittest

def dividir(a, b):

    if b == 0:
        raise ZeroDivisionError("Erro!")

    return a / b

class TestErro(unittest.TestCase):

    def test_divisao_por_zero(self):
        with self.assertRaises(ZeroDivisionError):
            dividir(10, 0)

if __name__ == "__main__":
    unittest.main(argv=[''], exit=False)
```

6 Gabarito — Desafio Final

```
import unittest

def calcular(numero):

    if numero == 0:
        raise ZeroDivisionError("Não pode dividir por zero.")

    return 100 / numero

def main():

    try:
        nome = input("Digite seu nome: ")

        idade = int(input("Digite sua idade: "))

        if idade < 0:
            raise ValueError("Idade inválida.")

        numero = int(input("Digite seu número favorito: "))

        resultado = calcular(numero)

    except ValueError:
        print("Erro: entrada inválida.")
    except ZeroDivisionError as erro:
        print("Erro:", erro)
    else:
        print(f"{nome}, resultado: {resultado}")

class TestCalculo(unittest.TestCase):

    def test_calculo_normal(self):
        self.assertEqual(calcular(2), 50)

    def test_divisao_zero(self):
        with self.assertRaises(ZeroDivisionError):
            calcular(0)

if __name__ == "__main__":
    unittest.main(argv=[''], exit=False)
```

7 Mini Projeto Final — Sistema de Cadastro Seguro (Nível Prova)

Neste projeto, vamos construir um sistema de cadastro com:

- Validação de dados
- Tratamento de exceções
- Uso de classes
- Testes automatizados

7.1 Objetivo do sistema

O sistema deve:

- Cadastrar usuários (nome e idade)
- Impedir idade inválida
- Permitir listar usuários
- Buscar usuário pelo nome

7.2 Arquivo principal (sistema.py)

```
class Usuario:

    def __init__(self, nome, idade):

        if idade < 0:
            raise ValueError("Idade não pode ser negativa.")

        self.nome = nome
        self.idade = idade

    def __str__(self):
        return f"{self.nome} ({self.idade} anos)"

class SistemaCadastro:

    def __init__(self):
        self.usuarios = []

    def cadastrar(self, nome, idade):

        usuario = Usuario(nome, idade)
        self.usuarios.append(usuario)

    def listar(self):

        if not self.usuarios:
            print("Nenhum usuário cadastrado.")
            return

        for u in self.usuarios:
            print(u)
```

```

def buscar(self, nome):

    for u in self.usuarios:
        if u.nome == nome:
            return u

    raise ValueError("Usuário não encontrado.")

```

7.3 Interface com tratamento de erros (main.py)

```

from sistema import SistemaCadastro

def menu():

    sistema = SistemaCadastro()

    while True:

        print("\n1 - Cadastrar")
        print("2 - Listar")
        print("3 - Buscar")
        print("0 - Sair")

        opcao = input("Escolha: ")

        try:

            if opcao == "1":

                nome = input("Nome: ")
                idade = int(input("Idade: "))

                sistema.cadastrar(nome, idade)

                print("Usuário cadastrado!")

            elif opcao == "2":

                sistema.listar()

            elif opcao == "3":

                nome = input("Nome: ")
                usuario = sistema.buscar(nome)

                print("Encontrado:", usuario)

            elif opcao == "0":

                print("Saindo...")
                break

        else:

            print("Opção inválida.")

```

```

        except ValueError as erro:
            print("Erro:", erro)

        except Exception:
            print("Erro inesperado.")

if __name__ == "__main__":
    menu()

```

7.4 Testes automatizados (test_sistema.py)

```

import unittest
from sistema import SistemaCadastro, Usuario

class TestSistema(unittest.TestCase):

    def setUp(self):
        self.sistema = SistemaCadastro()

    def test_cadastro_usuario(self):

        self.sistema.cadastrar("Ana", 18)

        self.assertEqual(len(self.sistema.usuarios), 1)

    def test_idade_invalida(self):

        with self.assertRaises(ValueError):
            Usuario("Ana", -5)

    def test_buscar_usuario(self):

        self.sistema.cadastrar("Carlos", 20)

        usuario = self.sistema.buscar("Carlos")

        self.assertEqual(usuario.nome, "Carlos")

    def test_buscar_inexistente(self):

        with self.assertRaises(ValueError):
            self.sistema.buscar("João")

if __name__ == "__main__":
    unittest.main(argv=[''], exit=False)

```

7.5 Exemplo de uso

```

1 - Cadastrar
2 - Listar
3 - Buscar
0 - Sair

```

Escolha: 1

Nome: Ana
Idade: 18
Usuário cadastrado!

Escolha: 2
Ana (18 anos)

Escolha: 3
Nome: Ana
Encontrado: Ana (18 anos)

7.6 O que este projeto trabalha

- Classes e objetos
- Métodos e atributos
- Tratamento de exceções com `try/except`
- Uso de `raise`
- Testes com `unittest`
- Organização em múltiplos arquivos

7.7 Desafios extras (nível hard)

- Impedir nomes duplicados
- Adicionar atualização de usuário
- Remover usuário
- Salvar dados em arquivo
- Criar testes para novos métodos